Especificación de requisitos de software (SRS)

Proyecto: SaborExpress | **Fecha de corte:** 14 de octubre de 2026 | **Versión:** 2.0 | **Tipo:** Basada en ISO/IEC/IEEE 29148:2018

Estado de la evidencia: inspección documental y estática del código. Las pruebas de ejecución y los datos institucionales no aportados no se presumen realizados.

1. Propósito, alcance y lectores

SaborExpress es una solución de apoyo a operaciones gastronómicas con API central, portal administrativo web y aplicación móvil. Este documento traduce capacidades reconocibles del repositorio a requisitos verificables; no constituye aceptación contractual ni certificación del sistema. Destinatarios: equipo de desarrollo, analista, instructor, QA, operación y responsables de producto.

2. Contexto y límites

Comprende registro/autenticación, gestión de sedes, personal, productos, pedidos, pagos, repartos, notificaciones y administración. Integra Wompi mediante endpoints de inicialización, conciliación y webhook; depende de red y credenciales de servicios externos. No se confirma en estos repositorios una aplicación de escritorio independiente, despliegue productivo estable ni certificación bancaria.

Actores

Gerente, administrador de sede, cliente, cajero, cocinero, mesero y repartidor. Los nombres exactos de roles y las asignaciones de permisos deben confrontarse con Shared/Constants/RoleNames y los seeders. El proveedor Wompi participa como sistema externo.

3. Requisitos funcionales

Convención: la columna «evidencia» significa que el código contiene un punto de entrada o pantalla; la columna no afirma que el caso de uso pasó una prueba funcional.

ID

Actor

Comportamiento requerido

Evidencia inicial

Prioridad

Verificación

RF-01

Usuario

Iniciar sesión con credenciales

POST /api/Auth/login

Alta

API + cliente de login

RF-02

Usuario

Recuperar contraseña mediante código y restablecimiento

AuthController

Alta

API

RF-03

Administrador

Administrar permisos y asignación de roles

Controllers/Roles,Permissions,UserRoles

Alta

API

RF-04

Gerente

Registrar y actualizar sedes

BranchesController

Alta

API

RF-05

Administrador

Consultar su sede asignada

GET /api/Branches/mine

Alta

API

RF-06

Administrador

Gestionar empleados

EmployeesController

Alta

API

RF-07

Personal

Consultar horarios y turnos

EmployeeSchedulesController

Media

API y cliente

RF-08

Administrador

Gestionar categorías y productos

ProductsController/CategoriesController

Alta

API

RF-09

Cliente

Ver productos por sede o categoría

GET /api/Products

Alta

API

RF-10

Cliente/cajero

Crear pedidos con detalle de productos

POST /api/Orders

Alta

API

RF-11

Personal

Filtrar pedidos por mesa, cliente y sede

GET /api/Orders/…

Alta

API

RF-12

Personal

Actualizar estado de pedido con autorización

PATCH /api/Orders/{id}/status

Alta

API

RF-13

Cliente

Cancelar pedido propio según restricciones

PATCH /api/Orders/{id}/cancel-mine

Alta

API

RF-14

Cajero

Registrar y consultar pagos por turno

PaymentsController

Alta

API

RF-15

Cliente

Iniciar pago digital Wompi

POST /api/Payments/wompi/init

Alta

API

RF-16

Proveedor de pago

Informar resultado por webhook autenticado

POST /api/Payments/wompi/webhook

Crítica

API; validar firma

RF-17

Cajero

Iniciar cobro con QR Wompi

POST /api/Payments/wompi/cashier-init

Media

API

RF-18

Gerente

Asignar repartidor a entrega

POST /api/Deliveries

Alta

API

RF-19

Repartidor

Actualizar estado de entrega autorizado

PATCH /api/Deliveries/{id}/status

Alta

API

RF-20

Cliente

Calificar reparto asociado

POST /api/Deliveries/{id}/rating

Media

API

RF-21

Usuario

Consultar sus notificaciones

NotificationsController

Media

API y cliente

RF-22

Cocina

Visualizar tablero de pedidos en preparación

PublicKitchenController y componente KitchenBoardPublic

Alta

API y móvil

RF-23

Cliente

Consultar sede más cercana por coordenadas

GET /api/Branches/nearest

Media

API

RF-24

Personal

Gestionar mesas, pedidos y menú diario

Tables,Orders,DailyMenu Controllers

Alta

API

RF-25

Administrador

Consultar reportes operativos web

src/app/reports/page.tsx

Media

Página identificada; funcionalidad a probar

4. Requisitos no funcionales

Los criterios siguientes son objetivos de aceptación propuestos, no mediciones efectuadas.

ID

Calidad

Criterio

Cómo verificar

RNF-01

Seguridad

Autorización por rol y por recurso (sede, propietario) en cada operación sensible

0 accesos cruzados en batería negativa

RNF-02

Autenticación

JWT con expiración, validación, rotación o revocación según diseño

Pruebas token vencido/manipulado

RNF-03

Pagos

Webhook con validación criptográfica, idempotencia y rastreo de referencia

Reenvío idéntico no duplica abono

RNF-04

Integridad

Transacciones coherentes para pedido/pago/entrega

Sin estados contradictorios en escenarios de error

RNF-05

Rendimiento

Definir carga nominal, p50/p95 HTTP y SignalR

Medir con entorno versionado; sin umbral inventado

RNF-06

Disponibilidad

Recuperación de caídas de API y base de datos

Ensayos de fallo y recuperación

RNF-07

Portabilidad

Instalación reproducible en entorno de desarrollo y Docker

Documentar variables y dependencias

RNF-08

Mantenibilidad

Organización modular controller/service/repository

Revisión por pares + checks estáticos

RNF-09

Compatibilidad

Ensayar proveedor relacional realmente elegido

Migraciones y CRUD pasan en motor escogido

RNF-10

Usabilidad

Tareas críticas de cliente, cajero y repartidor

Prueba con participantes y tasa de éxito

RNF-11

Accesibilidad

Etiquetas, navegación por teclado y contraste adecuados

Auditoría automática + revisión manual

RNF-12

Privacidad

Minimización de datos y no exposición de claves

Escaneo de secretos + políticas de acceso

RNF-13

Observabilidad

Errores trazables sin credenciales ni datos financieros

Correlación de logs por transacción

RNF-14

Internacionalización

Mensajes consistentes con preferencias del usuario

Pruebas idioma/locale

RNF-15

Respaldo

Copias y restauración comprobadas del proveedor seleccionado

Ensayo aislado documentado

5. Reglas de negocio y restricciones

Un pedido se vincula a una sede; puede provenir de cliente, empleado o mesa y opcionalmente asociarse a una dirección de entrega.

Order.CustomerId, EmployeeId, TableId y AddressId permiten escenarios distintos. El servicio debe validar la coherencia según tipo de pedido.

Un detalle identifica producto, cantidad, precio unitario, subtotal y estado; la edición del detalle deja trazabilidad histórica cuando corresponda.

Un pago pertenece a un pedido. CashierId puede ser nulo en la integración digital directa.

El controlador de pedidos contempla una restricción para no confirmar manualmente pedidos con intentos recientes de Wompi pendientes. El comportamiento completo exige prueba de servicio y webhook.

El acceso de empleados y repartidores depende de roles, sede y en algunas acciones de la política ActiveShift.

Productos con BranchId nulo se interpretan en el modelo como globales; los asignados a sede se restringen a esa sede.

El flujo de cambios sensibles debe considerar auditoría, permisos y validación de propiedad del recurso.

6. Interfaces y dependencias

Cliente web: Next.js 16, React 19. Cliente móvil: Expo 57, React Native 0.86. API: ASP.NET Core 8 con JSON, JWT Bearer y SignalR. Persistencia: EF Core y motor seleccionable MySQL, SQL Server o PostgreSQL. Servicios externos: Wompi, correo y geocodificación donde el despliegue los configure.

7. Escenarios de aceptación por flujo

Pedido de cliente: autenticarse, consultar catálogo por sede, seleccionar productos, confirmar tipo/dirección, registrar pedido, revisar importe y consultar estado. Resultado esperado: un solo pedido, suma calculada correctamente, vínculo a sede y autorización del cliente.

Pago digital: iniciar transacción con referencia única, procesar estado en sandbox, aceptar webhooks válidos, rechazar firma inválida, tolerar eventos duplicados. Resultado esperado: sin doble acreditación ni confirmación anticipada.

Repartidor: consultar solo entregas permitidas, actualizar estado, impedir modificar reparto de otra sede, permitir calificación de cliente autorizado.

8. Control de cambios y aceptación

Toda historia debe registrar identificador, versión, fuente, responsable, dependencias, prueba positiva y negativa, evidencia anonimizada, resultado y aprobación. Documentos de diseño y endpoints derivan de estos requisitos mediante la matriz de trazabilidad incluida en 9-traceability.

Fuentes primarias de implementación

API: https://github.com/yersonct/SaborExpressBack/tree/main

Web: https://github.com/yersonct/SaborExpressWeb/tree/main

Móvil: https://github.com/yersonct/SaborExpressMovil/tree/main

Referencia de presentación: https://github.com/jsgh1/SRD-v1/tree/dev/docs/docs-SRD

Control documental: documento elaborado a partir del estado observado en octubre de 2026. Antes de entregar, verificar el commit exacto, responsable, revisión del instructor y cualquier cambio posterior del código.